

Spółeczna Akademia Nauk

Instytut Technologii Informatycznych

Kierunek studiów:

INFORMATYKA

Przedmiot:

Systemy Szkieletowe

System rezerwacji sal

Maura Wasielewska

Nr albumu: 121466

Grupa: 2

Prowadzący:

mgr inż. Krystian Gumiński

Łódź 2026

Spis treści

1	Wstęp	2
2	Cel projektu	3
3	Przegląd istniejących rozwiązań	4
3.1	Microsoft Outlook	4
3.2	Microsoft Bookings	4
3.3	Dedykowane systemy rezerwacji sal	4
4	Opis architektury	5
5	Opis implementacji	6
5.1	Frontend	6
5.2	Backend	6
5.3	Baza Danych	7
5.4	Docker	8
5.5	Testy	8
5.6	Monitoring	8
6	Podsumowanie i wnioski	10
	Bibliografia	13

1 Wstęp

W ramach projektu została opracowana aplikacja internetowa służąca do rezerwacji sal. System umożliwia użytkownikom przeglądanie dostępnych pomieszczeń, wyszukiwanie sal oraz dokonywanie rezerwacji w określonym terminie. Aplikacja została zaprojektowana jako system składający się z części frontendowej, backendowej oraz bazy danych. W projekcie wykorzystano język Python i framework Flask, natomiast interfejs użytkownika wykonano przy użyciu HTML, CSS i JavaScript. Dane przechowywane są w bazie SQLite z wykorzystaniem SQLAlchemy.

2 Cel projektu

Celem projektu jest stworzenie prostego systemu umożliwiającego sprawne zarządzanie rezerwacjami sal. System pozwala użytkownikowi na rejestrację, logowanie, przeglądanie i wyszukiwanie sal, sprawdzanie ich dostępności oraz tworzenie i anulowanie rezerwacji. Dodatkowo użytkownik może korzystać z kalendarza oraz sprawdzać zajętość pomieszczeń. Drugim istotnym celem było zastosowanie mechanizmu kontroli konfliktów terminów, który uniemożliwia zarezerwowanie tej samej sali przez dwóch użytkowników w nakładających się godzinach. System posiada również konto administratora, który może zarządzać dostępnymi salami.

3 Przegląd istniejących rozwiązań

Na rynku dostępnych jest wiele systemów umożliwiających rezerwację pomieszczeń. Są one wykorzystywane między innymi w przedsiębiorstwach, szkołach i uczelniach.

3.1 Microsoft Outlook

Microsoft Outlook umożliwia planowanie spotkań oraz rezerwowanie dostępnych zasobów, w tym sal konferencyjnych. System jest zintegrowany z kalendarzem i pozwala użytkownikom sprawdzać dostępność wybranych zasobów. W porównaniu z projektowanym systemem Outlook oferuje znacznie szerszy zakres funkcji związanych z komunikacją i organizacją pracy.

3.2 Microsoft Bookings

Microsoft Bookings jest rozwiązaniem umożliwiającym zarządzanie terminami oraz rezerwacjami. System pozwala na określanie dostępnych terminów i zarządzanie rezerwacjami za pomocą interfejsu internetowego. Projektowany system jest rozwiązaniem prostszym i skoncentrowanym wyłącznie na rezerwacji sal.

3.3 Dedykowane systemy rezerwacji sal

Na rynku dostępne są również dedykowane systemy przeznaczone do zarządzania przestrzenią i rezerwacją pomieszczeń. Oferują one między innymi kalendarze, zarządzanie użytkownikami oraz sprawdzanie dostępności sal. Projektowany system realizuje podstawowe funkcje tego typu aplikacji, jednocześnie zachowując prostą strukturę i niewielką liczbę wymaganych komponentów.

4 Opis architektury

System został podzielony na trzy podstawowe warstwy: frontend, backend oraz bazę danych. Frontend odpowiada za prezentację informacji oraz komunikację z użytkownikiem. Backend realizuje logikę aplikacji, obsługuje żądania HTTP i kontroluje dostęp do poszczególnych funkcji. Baza danych przechowuje informacje o użytkownikach, salach oraz rezerwacjach.

Tabela 1: Główne komponenty architektury systemu

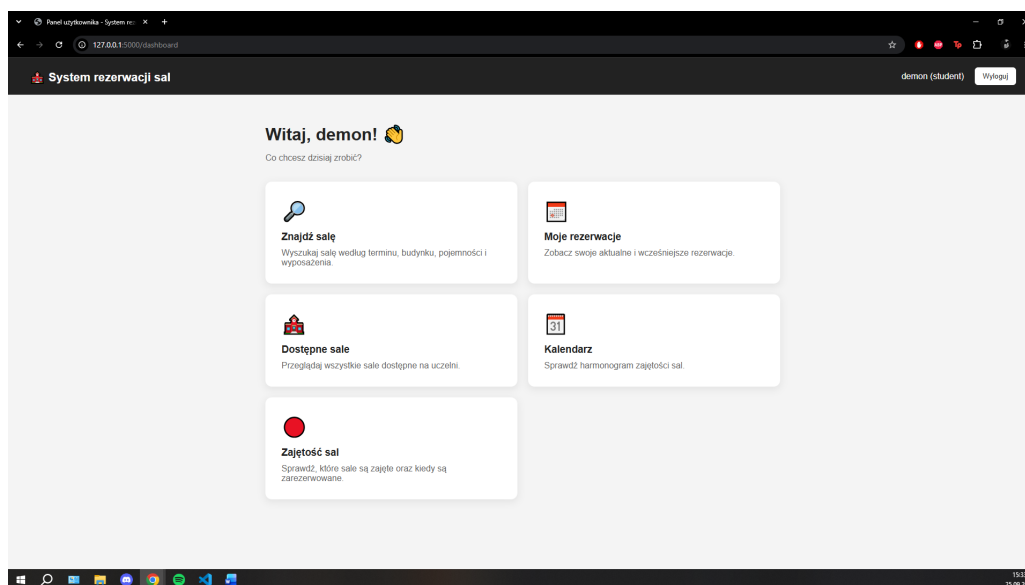
Lp.	Komponent	Warstwa
1	Frontend	Prezentacja i obsługa użytkownika
2	Backend Flask	Logika aplikacji i API
3	SQLAlchemy	Komunikacja z bazą danych
3	SQLite	Przechowywanie danych
3	Docker	Uruchamianie aplikacji w kontenerze

5 Opis implementacji

W rozdziale przedstawiono najważniejsze elementy implementacji systemu. Opis został podzielony na frontend, backend, bazę danych oraz elementy związane z uruchamianiem, testowaniem i monitorowaniem aplikacji.

5.1 Frontend

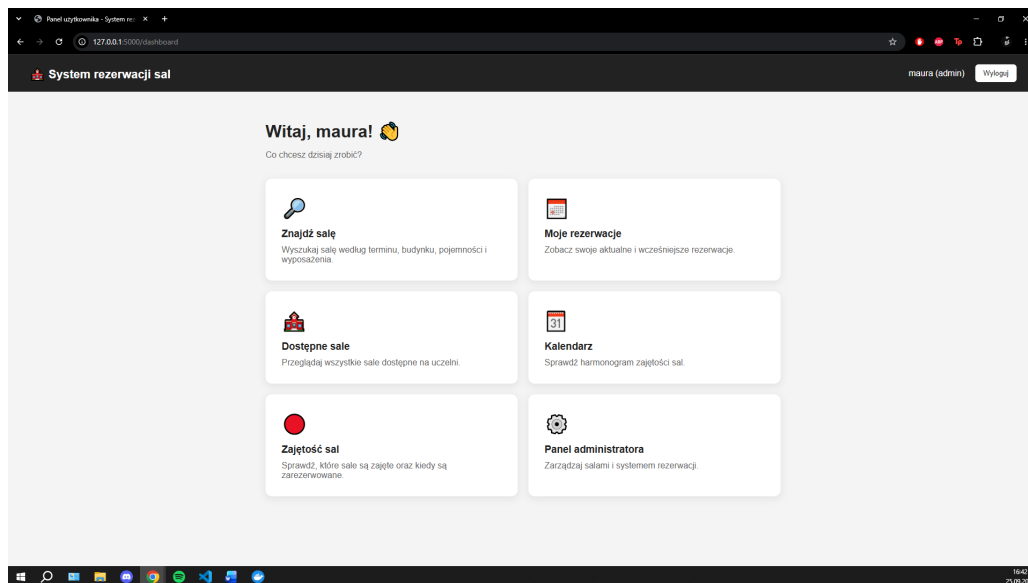
Frontend aplikacji został wykonany przy wykorzystaniu HTML, CSS oraz JavaScript. HTML odpowiada za strukturę poszczególnych stron, CSS za wygląd interfejsu, natomiast JavaScript umożliwia komunikację z backendem i dynamiczne wyświetlanie danych. Aplikacja posiada między innymi strony logowania i rejestracji, listę sal, wyszukiwarke, formularz rezerwacji, listę własnych rezerwacji, kalendarz, widok zajętości sal oraz panel administratora.



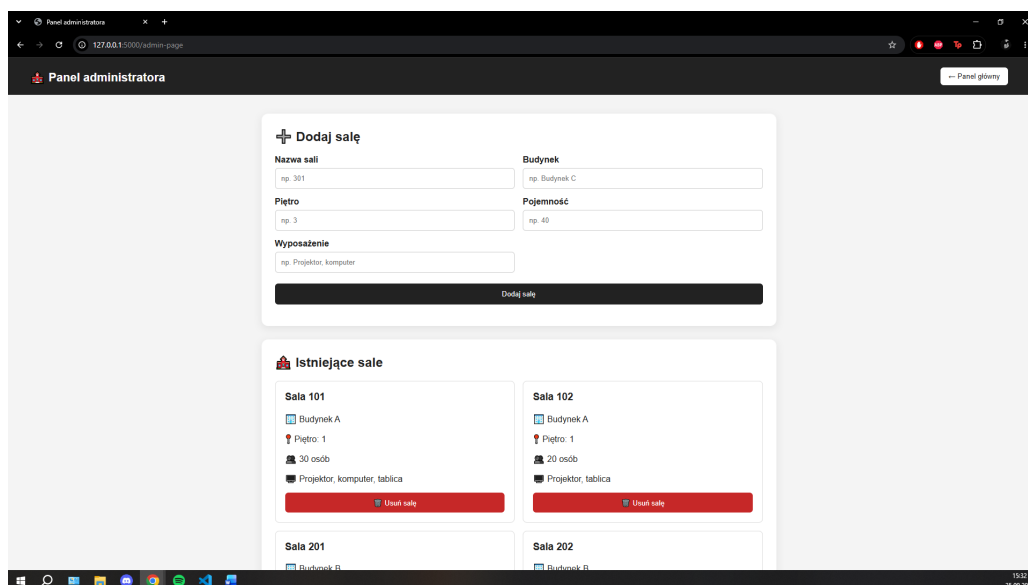
Rysunek 1: Panel główny użytkownika systemu.

5.2 Backend

Backend został wykonany w języku Python z wykorzystaniem frameworka Flask. Odpowiada on za obsługę użytkowników, sal i rezerwacji oraz komunikację z bazą danych. Backend udostępnia endpointy wykorzystujące między innymi metody HTTP GET, POST i DELETE. Podczas tworzenia rezerwacji system sprawdza poprawność danych oraz dostępność wybranego terminu. Jeżeli istnieje konflikt z inną rezerwacją, operacja zostaje odrzucona. Backend odpowiada również za kontrolę uprawnień. Funkcje administracyjne są dostępne wyłącznie dla użytkownika posiadającego rolę administratora.



Rysunek 2: Panel administratora systemu.



Rysunek 3: Formularz dodawania sali przez administratora.

5.3 Baza Danych

Do przechowywania danych wykorzystano bazę SQLite. Komunikacja pomiędzy aplikacją a bazą danych realizowana jest przy pomocy biblioteki SQLAlchemy. W bazie przechowywane są informacje dotyczące użytkowników, sal oraz rezerwacji. Model użytkownika zawiera między innymi identyfikator, imię, adres e-mail, hasło oraz rolę. Model sali przechowuje nazwę sali, budynek, piętro, pojemność, wyposażenie oraz status. Model rezerwacji zawiera informacje o użytkowniku, sali, dacie oraz godzinach rozpoczęcia i zakończenia rezerwacji. Hasła użytkowników są przechowywane w postaci zahasowanej.

5.4 Docker

Aplikacja została przygotowana do uruchamiania z wykorzystaniem technologii Docker. W projekcie znajduje się plik Dockerfile, który określa sposób przygotowania środowiska aplikacji. Wykorzystano obraz Python 3.11 oraz plik requirements.txt zawierający wymagane biblioteki.

Obraz aplikacji został utworzony za pomocą polecenia: `docker build -t system-rezerwacji-sal`

Zastosowanie Dockera pozwala na uruchomienie aplikacji w przygotowanym i powtarzalnym środowisku.

```
PS C:\Users\maura\Github\system-rezerwacji-sal> docker build -t system-rezerwacji-sal .
[+] Building 64.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 195B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 28
=> [1/4] FROM docker.io/library/python:3.11-slim@sha256:da047cb8f9d1d98e5c070f5300ba9f7274e33b8fc0e5be5ed88740aed1b95ba9
=> => resolve docker.io/library/python:3.11-slim@sha256:da047cb8f9d1d98e5c070f5300ba9f7274e33b8fc0e5be5ed88740aed1b95ba9
=> [internal] load build context
=> => transferring context: 13.21MB
=> CACHED [2/4] WORKDIR /app
=> [3/4] COPY . .
=> [4/4] RUN pip install --no-cache-dir -r requirements.txt
=> exporting to image
=> exporting layers
=> => exporting manifest sha256:b1f7e9a847d576815cfe6d670fd3252304be4902ba364bbbd619b381b72af8
=> => exporting config sha256:b6b2c74da91bd3ce7a992de9c7691b3d4a01fa6325e61baa90be9f9e9e43e
=> => exporting attestation manifest sha256:b14484e46ba93d5922ac80b244ede2d49f6b1806abdfb394c6d7fdec0e632f
=> => exporting manifest list sha256:ce6d1f0f16e3fb23d78a3ffe352335308dcaacd5821aab63de210a8be6cc71b
=> naming to docker.io/library/system-rezerwacji-sal:latest
=> => unpacking to docker.io/library/system-rezerwacji-sal:latest
```

Rysunek 4: Budowanie obrazu aplikacji za pomocą technologii Docker.

5.5 Testy

Do testowania aplikacji wykorzystano bibliotekę unittest. Przygotowane testy sprawdzają między innymi dostępność strony głównej, strony logowania oraz endpointu odpowiedzialnego za listę sal. Testy uruchamiane są za pomocą polecenia: `python -m unittest discover -s tests -v`. Wszystkie trzy przygotowane testy zakończyły się poprawnie: `Ran 3 tests ... OK`

```
PS C:\Users\maura\Github\system-rezerwacji-sal> python -m unittest discover -s tests -v
test_lista_sal (test_app.TestSystemRezerwacji.test_lista_sal) ... ok
test_strona_glowna (test_app.TestSystemRezerwacji.test_strona_glowna) ... C:\Users\maura\AppData\Local\Programs\Python\Python312\Lib\unittest\case.py:589: ResourceWarning: unclosed file <io.BufferedReader name='C:\Users\maura\Github\system-rezerwacji-sal\backend\..\frontend\index.html'>
  if method() is not None:
ResourceWarning: Enable tracemalloc to get the object allocation traceback
ok
test_strona_logowania (test_app.TestSystemRezerwacji.test_strona_logowania) ... C:\Users\maura\AppData\Local\Programs\Python\Python312\Lib\unittest\case.py:589: ResourceWarning: unclosed file <io.BufferedReader name='C:\Users\maura\Github\system-rezerwacji-sal\backend\..\frontend\login.html'>
  if method() is not None:
ResourceWarning: Enable tracemalloc to get the object allocation traceback
ok

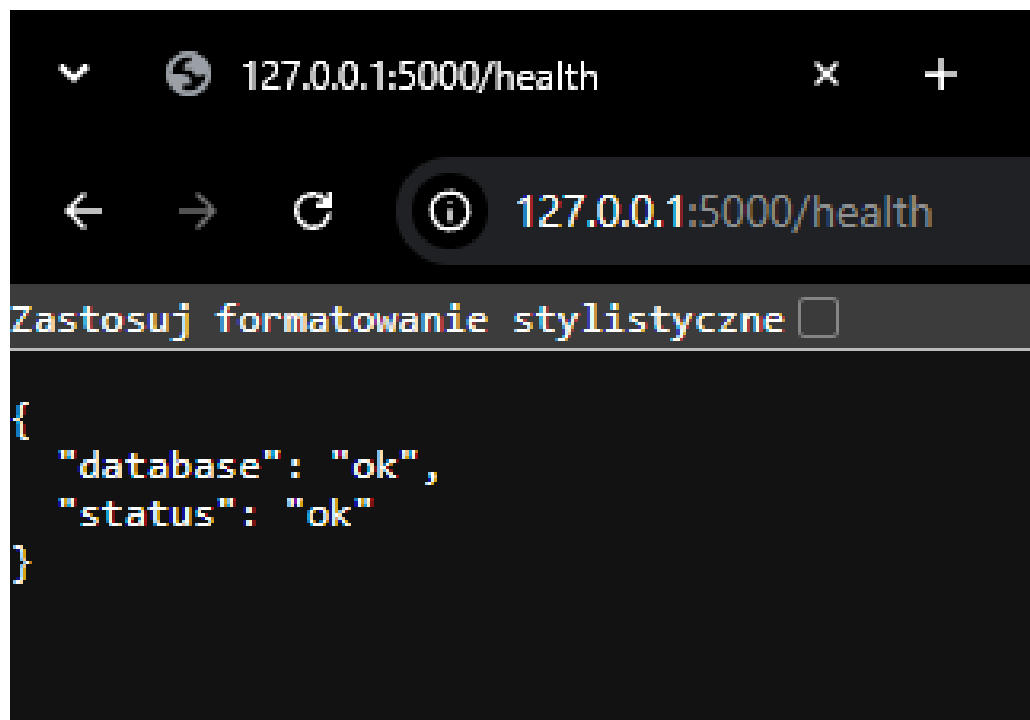
Ran 3 tests in 0.463s

OK
```

Rysunek 5: Wynik wykonania testów jednostkowych aplikacji.

5.6 Monitoring

W celu sprawdzania stanu aplikacji przygotowano endpoint: GET /health. Endpoint sprawdza działanie aplikacji oraz możliwość wykonania operacji na bazie danych. W przypadku prawidłowego działania zwracany jest status ok oraz informacja, że baza danych działa poprawnie. Mechanizm umożliwia szybkie sprawdzenie podstawowego stanu systemu.



Rysunek 6: Wynik działania endpointu monitorującego stan aplikacji.

6 Podsumowanie i wnioski

W ramach projektu utworzono internetowy system rezerwacji sal umożliwiający użytkownikom wyszukiwanie pomieszczeń, sprawdzanie ich dostępności oraz tworzenie i anulowanie rezerwacji. System posiada mechanizm zapobiegania konfliktom terminów oraz podział użytkowników na zwykłych użytkowników i administratorów. Administrator ma dodatkowo możliwość zarządzania dostępnymi salami. Do wykonania projektu wykorzystano Python, Flask, HTML, CSS, JavaScript, SQLite, SQLAlchemy oraz Docker. Kod projektu jest zarządzany przy użyciu Git i przechowywany na platformie GitHub. Projekt pozwolił na praktyczne wykorzystanie wiedzy związanej z tworzeniem aplikacji internetowych, projektowaniem baz danych, tworzeniem API, testowaniem oraz uruchamianiem aplikacji w środowisku kontenerowym

Spis tabel

1	Główne komponenty architektury systemu	5
---	--	---

Spis rysunków

1	Panel główny użytkownika systemu.	6
2	Panel administratora systemu.	7
3	Formularz dodawania sali przez administratora.	7
4	Budowanie obrazu aplikacji za pomocą technologii Docker.	8
5	Wynik wykonania testów jednostkowych aplikacji.	8
6	Wynik działania endpointu monitorującego stan aplikacji.	9

Bibliografia

- [1] M. Grinberg. *Flask Web Development: Developing Web Applications with Python*. 2 wyd. O'Reilly Media, 2018.
- [2] Pallets Projects. *Flask Documentation*. 2026. URL: <https://flask.palletsprojects.com/>.
- [3] Python Software Foundation. *Python Documentation*. 2026. URL: <https://docs.python.org/3/>.
- [4] SQLAlchemy Authors. *SQLAlchemy Documentation*. 2026. URL: <https://docs.sqlalchemy.org/>.
- [5] SQLite Consortium. *SQLite Documentation*. 2026. URL: <https://www.sqlite.org/docs.html>.
- [6] Docker Inc. *Docker Documentation*. 2026. URL: <https://docs.docker.com/>.
- [7] Python Software Foundation. *unittest – Unit testing framework*. 2026. URL: <https://docs.python.org/3/library/unittest.html>.
- [8] Microsoft. *Microsoft Outlook*. 2026. URL: <https://www.microsoft.com/microsoft-365/outlook/outlook-for-business>.
- [9] Microsoft. *Microsoft Bookings*. 2026. URL: <https://www.microsoft.com/microsoft-365/business/scheduling-and-booking-app>.